This was made possible by keeping a trie, and for each key pressed refining the search.

1. When keypad button 2 is pressed, we would start traversing the trie, going, in parallel, to the subtrees linked by edges marked with `'a'`, `'b'`, `'c'` (all three of them would likely be in the trie for any language using Latin alphabet).
2. When the second keypad button is pressed, for each of the three paths that we are currently traversing, T9 checks if they have children labeled with `'m'`, `'n'` and `'o'`, and keep track of the nodes reached at this second level. Each combination represents a path from the root to a level-2 node. Most likely, not all of the 9 combinations would have a path in the trie, for example it's unlikely any word will start with "bn".
3. The process continues with the next buttons pressed, till there is no node reachable through the possible paths traversed.

For this specific task, the trie nodes would likely store more than just a Boolean for each key: they would rather store the corpus frequency of that word (for instance, how likely it is that a word is used in English). This way, the most likely result would be returned, when more than one result is available. For instance, you would somehow expect that `"and"` would be preferred over `"cod"`.

6.4.5 Autocomplete

In the last 10 years or so, we have all become familiar with the autocomplete feature of search boxes. Even more, we